

LAB-4: Priklausomybiu Izoliavimas naudojant Fake ir Stub

Situacija

Isivaizduokite, kad jusu sistema jau veikia. Turite StudentService, kuris iesko studentu, validuoja duomenis ar skaiciuoja vidurkius. Taciau realiame projekte atsiranda problema: viena komanda dar nebaige savo modulio, API kartais neveikia, arba implementacija dar bus keiciama.

Klausimas: ar jusu verslo logika turi laukti kitu moduliu? Ar galite dirbti nepriklausomai?

Tikslas

Isplesti LAB-3 architektura taip, kad sistema butu izoliuota nuo konkreciu implementaciju ir leistu kontroliuoti priklausomybiu elgesi per interfeisus.

Bendri reikalavimai

- Program.cs dirba tik su interface.
- Jokios verslo logikos Main metode.
- Jokia klase neturi pati kurti savo priklausomybes (new).
- Priklausomybes perduodamos per konstruktoriu.
- Sukurtos Fake ir Stub implementacijos.
- Pademonstruotos bent dvi logikos sakos.
- README paaiskina, kas buvo izoliuota ir kodel.

1 DALIS - Priklausomybes izoliavimas (Refactor)

Pasirinkite viena is savo LAB-3 strategiju ir patikrinkite, ar klase nepriklauso nuo konkretios implementacijos.

```
// Blogas pavyzdys
private IStudentFinder _finder = new FindById();
```

Jei randate toki koda - atlikite refactor ir perduokite priklausomybe per konstruktoriu.

2 DALIS - Fake (nebaigto modulio imitavimas)

Sukurkite Fake klase, kuri leidzia dirbti nepriklausomai nuo realios implementacijos. Ji turi implementuoti ta pati interface ir grazinti stabiliu rezultata.

```
public class FakeStudentFinder : IStudentFinder
{
    public Student? Find(Group g, string query)
    {
        return new Student(1, "Fake", "fake@test.com");
    }
}
```

3 DALIS - Stub (kontroliuojamas elgesys)

Sukurkite Stub klase, kuri leidzia valdyti rezultata per konstruktoriu ir pademonstruoti skirtingas verslo logikos sakas.

```
public class StubStudentFinder : IStudentFinder
{
    private readonly Student? _result;

    public StubStudentFinder(Student? result)
    {
        _result = result;
    }

    public Student? Find(Group g, string query)
    {
        return _result;
    }
}
```

4 DALIS - Verslo logikos stabilumas

Pakeiskite Fake i Stub ir pakeiskite Stub parametra. Verslo logikos klase neturi buti modifikuojama. Tik perduodama kita implementacija.

5 DALIS (Challenge) - Runtime pasirinkimas

Leiskite runtime metu pasirinkti: realia implementacija, Fake arba Stub. Trumpai paaiskinkite README faile, kodėl tokia architektūra naudinga didesnėse sistemose.

Dazniausios klaidos

- new naudojamas klases viduje vietoj konstruktoriaus. - Fake ir Stub naudojami be aiskaus tikslo. - Verslo logika lieka Main metode. - StudentService modifikuojamas vietoj priklausomybes pakeitimo.
- Nera architekturinio paaiskinimo README faile.

Vertinimas (10 balu)

2 balai - Teisingas priklausomybes izoliavimas 2 balai - Fake implementacija 2 balai - Stub implementacija 2 balai - Aiskiai pademonstruotos logikos sakos 2 balai - README su architekturiniais paaiskinimais